

## 14.3 Índices multinivel dinámicos utilizando árboles B y B<sup>+</sup>

Los árboles B y B<sup>+</sup> son casos especiales de estructuras de datos en forma de árbol bien conocidas, de cuya terminología hacemos una introducción. Un **árbol** está formado por **nodos**. Cada nodo del árbol, excepto el nodo especial denominado **raíz**, tiene un nodo **padre** y varios (ninguno o más) nodos **hijo**. El nodo raíz no tiene padre. Un nodo que no tiene ningún nodo hijo se llama nodo **hoja**; un nodo que no es hoja es un nodo **interno**. El **nivel** de un nodo siempre es uno más que el nivel de su padre, siendo cero el nivel del nodo raíz.<sup>5</sup> Un **subárbol** de un nodo consta de ese nodo y de todos sus nodos **descendientes** (sus nodos hijo, los nodos hijo de sus nodos hijo, etcétera). Una definición recursiva precisa de subárbol es que consta de un nodo  $n$  y de los subárboles de todos los nodos hijo de  $n$ . La Figura 14.7 ilustra una estructura de datos en árbol. En esta figura, el nodo raíz es A, y sus nodos hijo son B, C y D. Los nodos E, J, C, G, H y K son nodos hoja.

Normalmente, mostramos un árbol con el nodo raíz en la parte superior (véase la Figura 14.7). Una forma de implementar un árbol es con tantos punteros en cada nodo como nodos hijo tiene ese nodo. En algunos casos, también se almacena un puntero padre en cada nodo. Además de los punteros, un nodo normalmente contiene algún tipo de información almacenada. Cuando un índice multinivel se implementa como una estructura en árbol, esa información incluye los valores del campo de indexación del fichero que se utiliza para guiar la búsqueda de un determinado registro.

En la Sección 14.3.1 introducimos los árboles de búsqueda y después explicamos los árboles B, que pueden utilizarse como índices multinivel dinámicos para guiar la búsqueda de los registros en un fichero de datos. Los nodos de un árbol B se mantienen llenos entre un 50% y un 100%, y los punteros a los bloques de datos se almacenan tanto en los nodos internos como en los nodos hoja de la estructura de árbol B. En la Sección 14.3.2 explicamos los árboles B<sup>+</sup>, una variación de los árboles B en los que los punteros a los bloques de datos de un fichero se almacenan únicamente en los nodos hoja, lo que puede llevar a índices de menos niveles y mayor capacidad.

### 14.3.1 Árboles de búsqueda y árboles B

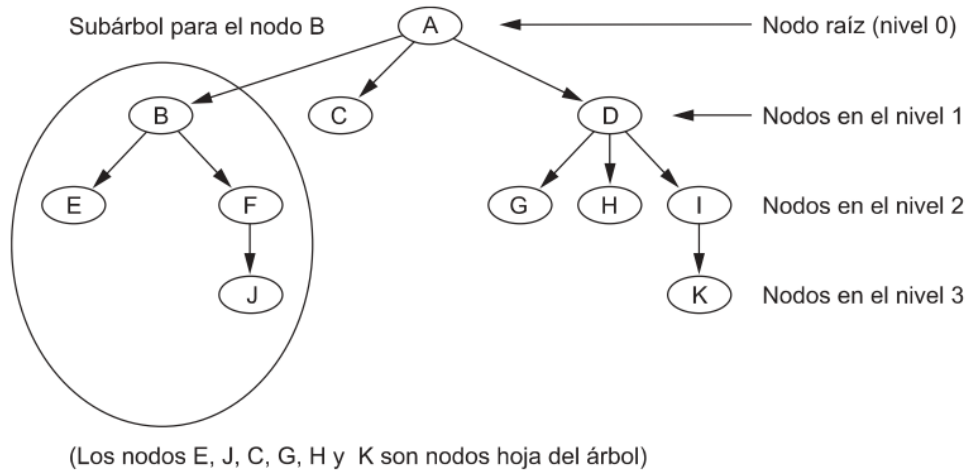
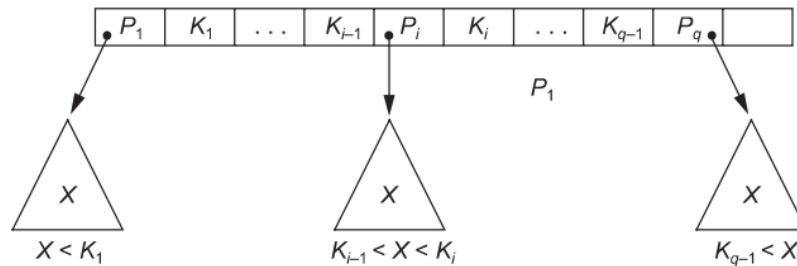
Un árbol de búsqueda es un tipo especial de árbol que sirve para guiar la búsqueda de un registro, dado el valor de uno de sus campos. Los índices multinivel explicados en la Sección 14.2 pueden imaginarse como una variante de un árbol de búsqueda; cada nodo del índice multinivel puede tener hasta  $fo$  punteros y  $fo$  valores clave, donde  $fo$  es el índice *fan-out*. Los valores del campo de índice de cada nodo nos guía hasta el siguiente nodo, hasta que alcanzamos el bloque del fichero de datos que contiene el registro deseado. Siguiendo un puntero, restringimos nuestra búsqueda en cada nivel a un subárbol del árbol de búsqueda e ignoramos todos los nodos que no están en este subárbol.

**Árboles de búsqueda.** Un árbol de búsqueda es ligeramente distinto a un índice multinivel. Un árbol de búsqueda de orden  $p$  es un árbol tal que cada nodo contiene, a lo sumo,  $p-1$  valores de búsqueda y  $p$  punteros en el orden  $\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$ , donde  $q \leq p$ ; cada  $P_i$  es un puntero a un nodo hijo (o puntero NULL o nulo); y cada  $K_i$  es un valor de búsqueda proveniente de algún conjunto ordenado de valores. Se supone que todos los valores de búsqueda son únicos.<sup>6</sup> La Figura 14.8 ilustra un nodo de un árbol de búsqueda. Un árbol de búsqueda debe cumplir, en todo momento, las siguientes restricciones:

1. Dentro de cada nodo,  $K_1 < K_2 < \dots < K_{q-1}$

<sup>5</sup> Esta definición estándar del nivel de un nodo de un árbol, que utilizamos a lo largo de la Sección 14.3, es diferente de la que ofrecimos para los índices multinivel en la Sección 14.2.

<sup>6</sup> Esta restricción se puede relajar algo. Si el índice se encuentra en un campo que no es clave, pueden existir valores de búsqueda duplicados y pueden modificarse la estructura de nodos y las reglas de navegación del árbol.

**Figura 14.7.** Estructura de datos en árbol que muestra un árbol desequilibrado.**Figura 14.8.** Nodo de un árbol de búsqueda con punteros a los subárboles que tiene por debajo.

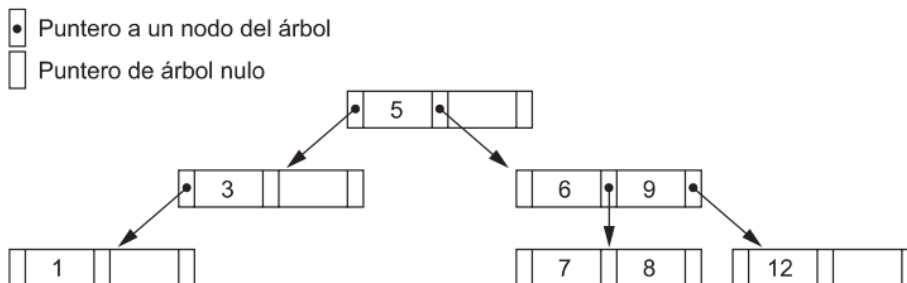
2. Para todos los valores de  $X$  del subárbol al cual apunta  $P_i$ , tenemos que  $K_{i-1} < X < K_i$  para  $1 < i < q$ ;  $X < K_i$  para  $i = 1$ ; y  $K_{q-1} < X$  para  $i = q$  (véase la Figura 14.8).

Siempre que buscamos un valor  $X$ , seguimos el puntero  $P_i$  apropiado, de acuerdo con las fórmulas de la condición 2. La Figura 14.9 ilustra un árbol de búsqueda de orden  $p = 3$  y valores de búsqueda enteros. Observe que algunos de los punteros  $P_i$  de un nodo pueden ser punteros nulos.

Podemos usar un árbol de búsqueda como mecanismo para buscar registros almacenados en un fichero de disco. Los valores del árbol pueden ser los valores de uno de los campos del registro, el llamado **campo de búsqueda** (que es el mismo que el campo de índice si un índice multinivel guía la búsqueda). Cada valor clave del árbol está asociado a un puntero de registro que tiene ese valor en el fichero de datos. Como alternativa, puede apuntar al bloque de disco que contiene ese registro. El árbol de búsqueda en sí puede almacenarse en disco, asignando cada nodo del árbol a un bloque del disco. Cuando se inserta un registro nuevo, es preciso actualizar el árbol de búsqueda incluyendo en él una entrada con el valor del campo de búsqueda del nuevo registro y un puntero a éste.

Para insertar valores de búsqueda en el árbol y eliminarlos, sin violar las dos restricciones anteriores, se utilizan algoritmos que, por regla general, no garantizan que el árbol de búsqueda esté equilibrado (es decir, que todas las hojas estén al mismo nivel).<sup>7</sup> El árbol de la Figura 14.7 no está equilibrado porque tiene nodos hoja en los niveles 1, 2 y 3. Es importante mantener equilibrados los árboles de búsqueda porque esto garantiza

<sup>7</sup> La definición de *equilibrado* es diferente para los árboles binarios. Los árboles binarios equilibrados se conocen como árboles AVL.

**Figura 14.9.** Árbol de búsqueda de orden  $p = 3$ .

que no habrá nodos en niveles muy profundos que requieran muchos accesos a bloques durante una búsqueda. Los árboles equilibrados procuran una velocidad de búsqueda uniforme independientemente del valor de la clave de búsqueda. Además, las eliminaciones de registros pueden hacer que queden nodos casi vacíos, con lo que hay un desperdicio de espacio importante y un aumento en el número de niveles. El árbol B hace frente a estos dos problemas especificando restricciones adicionales en el árbol de búsqueda.

**Árboles B.** El árbol B tiene restricciones adicionales que garantizan que el árbol siempre estará equilibrado y que el espacio desperdiciado por la eliminación, si lo hay, nunca será excesivo. Los algoritmos para insertar y eliminar son más complejos para poder mantener estas restricciones. No obstante, la mayor parte de las inserciones y eliminaciones son procesos simples que se complican sólo en circunstancias especiales (por ejemplo, siempre que intentamos una inserción en un nodo que ya está lleno o una eliminación de un nodo que está a menos de la mitad de su capacidad). De manera más formal, un árbol B de orden  $p$ , utilizado como estructura de acceso según un *campo clave* para buscar registros en un fichero de datos, se puede definir de este modo:

1. Cada nodo interno del árbol B (véase la Figura 14.10[a]) tiene la forma:

$$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle$$

donde  $q \leq p$ . Cada  $P_i$  es un **puntero de árbol** (un puntero a otro nodo del árbol B). Cada  $Pr_i$  es un **puntero de datos**<sup>8</sup> (un puntero al registro cuyo valor del campo clave de búsqueda es igual a  $K_i$  [o al bloque del fichero de datos que contiene ese registro]).

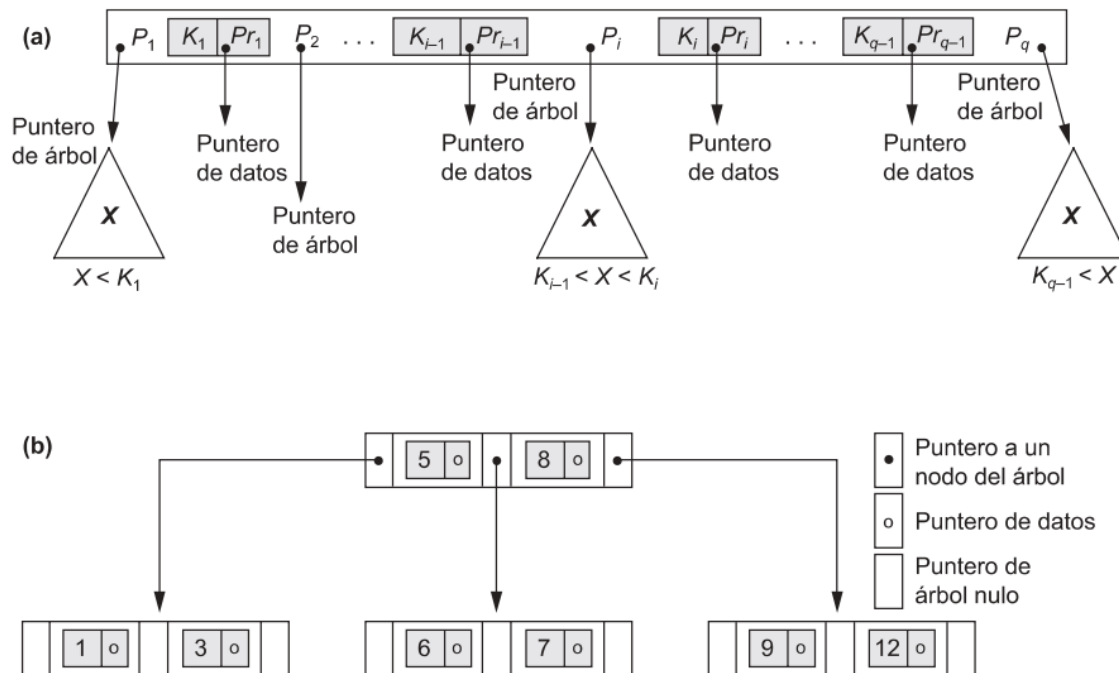
2. Dentro de cada nodo,  $K_1 < K_2 < \dots < K_{q-1}$ .
3. Para todos los valores del campo clave de búsqueda  $X$  del subárbol al que apunta  $P_i$  (el subárbol  $i$ ; véase la Figura 14.10[a]), tenemos:

$$K_{i-1} < X < K_i \text{ para } 1 < i < q; X < K_i \text{ para } i = 1; \text{ y } K_{i-1} < X \text{ para } i = q.$$

4. Cada nodo tiene a lo sumo  $p$  punteros de árbol.
5. Cada nodo, excepto los nodos raíz y hoja, tienen por lo menos  $\lceil (p/2) \rceil$  punteros de árbol. El nodo raíz tiene como mínimo dos punteros de árbol, a menos que sea el único nodo del árbol.
6. Un nodo con  $q$  punteros de árbol,  $q \leq p$ , tiene  $q - 1$  valores del campo clave de búsqueda (y, por tanto, tiene  $q - 1$  punteros de datos).
7. Todos los nodos hoja se encuentran en el mismo nivel. Los nodos hoja tienen la misma estructura que los internos, excepto que todos sus *punteros de árbol*  $P_i$  son nulos.

<sup>8</sup> Un puntero de datos es una dirección de bloque, o una dirección de registro; este último es esencialmente una dirección de bloque y un desplazamiento de registro dentro del bloque.

**Figura 14.10.** Estructuras de árbol B. (a) Nodo A en un árbol B con  $q - 1$  valores de búsqueda. (b) Un árbol B de orden  $p = 3$ . Los valores se insertaron en este orden: 8, 5, 1, 7, 3, 12, 9, 6.



La Figura 14.10(b) ilustra un árbol B de orden  $p = 3$ . Observe que todos los valores de búsqueda  $K$  del árbol B son únicos porque asumimos que el árbol se utiliza como una estructura de acceso en un campo clave. Si utilizamos un árbol B *en un campo que no es clave*, debemos cambiar la definición de los punteros de fichero  $Pr_i$  para que apunten a un bloque (o grupo de bloques) que contienen los punteros a los registros del fichero. Este nivel adicional de indirección es parecido a la Opción 3, explicada en la Sección 14.1.3, para los índices secundarios.

Un árbol B empieza con un solo nodo raíz (que también es un nodo hoja) en el nivel 0 (cero). Una vez que el nodo raíz está lleno con  $p - 1$  valores de clave de búsqueda e intentamos insertar otra entrada en el árbol, el nodo raíz se divide en dos nodos en el nivel 1. En el nodo raíz sólo queda el valor central, y el resto de valores se dividen uniformemente entre los otros dos nodos. Cuando un nodo no raíz está lleno y se inserta en él una entrada nueva, dicho nodo se divide en otros dos al mismo nivel, y la entrada central se mueve al nodo padre junto con dos punteros a los nuevos nodos divididos. Si el nodo padre está lleno, también es dividido. La división puede propagarse hasta llegar al nodo raíz, creando un nuevo nivel si se divide la raíz. Aquí no explicamos en profundidad los algoritmos para los árboles B; en cambio, en la siguiente sección hacemos una introducción de los procedimientos de búsqueda e inserción para los árboles B<sup>+</sup>.

Si la eliminación de un valor provoca que un nodo no llegue a la mitad de su capacidad, es combinado con sus nodos vecinos, algo que también puede propagarse hasta la raíz. Por tanto, la eliminación puede reducir el número de niveles del árbol. Con ayuda de análisis y simulaciones se ha demostrado que, después de numerosas inserciones y eliminaciones aleatorias en un árbol B, los nodos quedan ocupados aproximadamente al 69% de su capacidad cuando se estabiliza el número de valores en el árbol. Esto también es cierto con los árboles  $B^+$ . Si ocurre esto, la división y combinación de nodos sólo se dará raramente, por lo que la inserción y la eliminación son muy eficaces. Si el número de valores aumenta, el árbol se expandirá sin problemas (aunque podría darse la división de nodos, de modo que algunas inserciones tardarían algo más de tiempo). El Ejemplo 4 ilustra el cálculo del orden  $p$  de un árbol B almacenado en disco.



**Ejemplo 4.** Supongamos que el campo de búsqueda tiene una longitud de  $V = 9$  bytes, que el tamaño del bloque de disco es  $B = 512$  bytes, que un puntero de registro (datos) es  $P_r = 7$  bytes, y que un puntero de bloque es  $P = 6$  bytes. Cada nodo del árbol B puede tener *como máximo*  $p$  punteros de árbol,  $p - 1$  punteros de datos, y  $p - 1$  valores de campo clave de búsqueda (véase la Figura 14.10[a]). Deben encajar en un solo bloque de disco si cada nodo del árbol B corresponde a un bloque de disco. Por tanto, tenemos:

$$\begin{aligned}(p * P) + ((p - 1) * (P_r + V)) &\leq B \\(p * 6) + ((p - 1) * (7 + 9)) &\leq 512 \\(22 * p) &\leq 528\end{aligned}$$

Podemos elegir que  $p$  sea un valor mayor que satisfaga la desigualdad anterior, que da  $p = 23$  (no se elige  $p = 24$  debido a las razones que exponemos a continuación).

En general, un nodo de un árbol B puede contener información adicional que los algoritmos que manipulan el árbol necesitan, como el número de entradas  $q$  en el nodo y un puntero al nodo padre. Por tanto, antes de que hagamos el cálculo precedente para  $p$ , debemos reducir el tamaño del bloque para que pueda entrar dicha información. A continuación veremos cómo calcular el número de bloques y niveles para un árbol B.

**Ejemplo 5.** Supongamos que el campo de búsqueda del Ejemplo 4 es un campo clave desordenado, y que construimos un árbol B en ese campo. Los nodos del árbol B están al 60% de su capacidad. Cada nodo, por término medio, tendrá  $p * 0,69 = 23 * 0,69$  o aproximadamente 16 punteros y, por tanto, 15 valores de campo clave de búsqueda. El *fan-out medio* es  $fo = 16$ . Podemos empezar en la raíz y ver cuántos valores y punteros pueden existir, por término medio, en cada nivel subsiguiente:

|          |             |                 |                |
|----------|-------------|-----------------|----------------|
| Raíz:    | 1 nodo      | 15 entradas     | 16 punteros    |
| Nivel 1: | 16 nodos    | 240 entradas    | 256 punteros   |
| Nivel 2: | 256 nodos   | 3.840 entradas  | 4.096 punteros |
| Nivel 3: | 4.096 nodos | 61.440 entradas |                |

En cada nivel calculamos el número de entradas multiplicando el número total de punteros del nivel anterior por 15, la cantidad media de entradas de cada nodo. Por tanto, para un tamaño de bloque, tamaño de puntero y tamaño del campo clave de búsqueda dados, un árbol B de dos niveles almacena  $3.840 + 240 + 15 = 4.095$  entradas por término medio; un árbol B de tres entradas alberga 65.535 entradas por término medio.

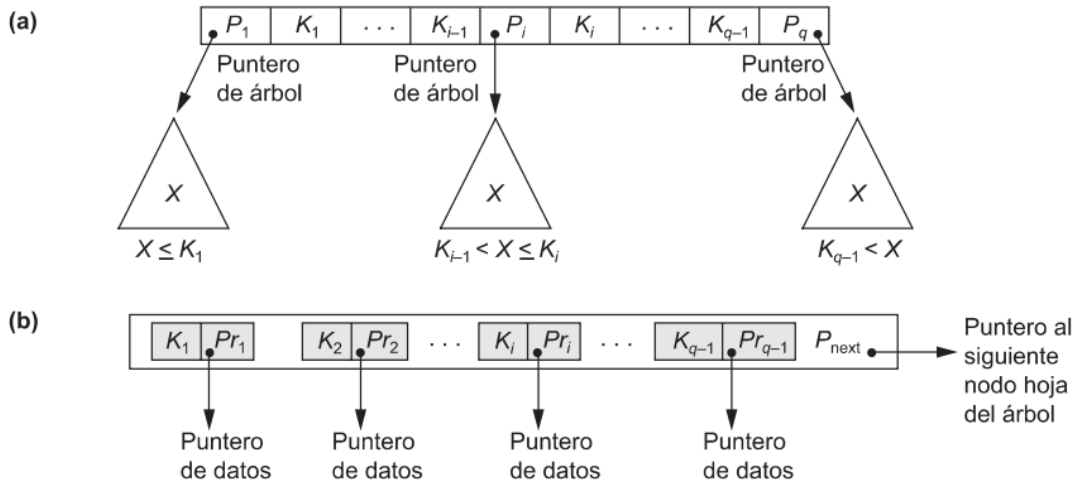
Los árboles B se utilizan a veces como organizaciones de fichero principales. En este caso, dentro de los nodos del árbol B se guardan los registros enteros, en lugar de entradas <clave de búsqueda, puntero de registro>. Esto funciona bien para ficheros con un *número relativamente pequeño de registros* y un *tamaño pequeño de registro*. En caso contrario, el *fan-out* y el número de niveles se hacen demasiado grandes para permitir un acceso eficaz.

En resumen, los árboles B proporcionan una estructura de acceso multinivel que es una estructura de árbol equilibrado en la que cada nodo está, como mínimo, medio lleno. Cada nodo de un árbol B de orden  $p$  puede tener como máximo  $p - 1$  valores de búsqueda.

### 14.3.2 Árboles B+

La mayoría de las implementaciones de un índice multinivel dinámico utilizan una variante de la estructura de datos en árbol B denominada **árbol B+**. En un árbol B, cada valor del campo de búsqueda aparece una vez en algún nivel del árbol, junto con un puntero de datos. En un árbol B<sup>+</sup> los punteros a datos se almacenan *sólo en los nodos hoja* del árbol, por lo cual, la estructura de los nodos hoja difiere de la de los nodos internos. Los nodos hoja tienen una entrada por cada valor del campo de indexación, junto con un puntero al registro (o al bloque que contiene ese registro) si el campo de búsqueda es un campo clave. En el caso de un campo de búsqueda que no es clave, el puntero apunta a un bloque que contiene punteros a los registros del fichero de datos, creándose un nivel extra de indirección.

**Figura 14.11.** Nodos de un árbol B<sup>+</sup>. (a) Nodo interno de un árbol B<sup>+</sup> con  $q - 1$  valores de búsqueda. (b) Nodo hoja de un árbol B<sup>+</sup> con  $q - 1$  valores de búsqueda y  $q - 1$  punteros de datos.



Los nodos hoja de un árbol B<sup>+</sup> normalmente están enlazados para ofrecer un acceso ordenado a los registros a través del campo de búsqueda. Estos nodos hoja son parecidos al primer nivel (nivel base) de un índice. Los nodos internos del árbol B<sup>+</sup> corresponden a los demás niveles de un índice multinivel. Algunos valores del campo de búsqueda de los nodo hoja se repiten en los nodos internos del árbol B<sup>+</sup> con el fin de guiar la búsqueda. La estructura de los *nodos internos* de un árbol B<sup>+</sup> de orden  $p$  (véase la Figura 14.11[a]) es la siguiente:

1. Cada nodo interno tiene esta forma:  
 $\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$   
donde  $q = p$  y cada  $P_i$  es un **puntero de árbol**.
2. Dentro de cada nodo interno,  $K_1 < K_2 < \dots < K_{q-1}$ .
3. Para todos los valores  $X$  del campo de búsqueda en el subárbol al cual apunta  $P_i$ , tenemos que  $K_{i-1} < X \leq K_i$  para  $1 < i < q$ ;  $X \leq K_i$  para  $i = 1$ ; y  $K_{q-1} < X$  para  $i = q$  (véase la Figura 14.11[a]).<sup>9</sup>
4. Cada nodo interno tiene, a lo sumo,  $p$  punteros de árbol.
5. Cada nodo interno, excepto el raíz, tiene al menos  $\lceil (p/2) \rceil$  punteros de árbol. El nodo raíz tiene como mínimo dos punteros de árbol si es un nodo interno.
6. Un nodo interno con  $q$  punteros,  $q \leq p$ , tiene  $q - 1$  valores del campo de búsqueda.

La estructura de los *nodos hoja* del árbol B<sup>+</sup> de orden  $p$  (véase la Figura 14.11[b]) es la siguiente:

1. Cada nodo hoja tiene la siguiente forma:  
 $\langle \langle K_1, Pr_1 \rangle, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_{sig} \rangle$   
donde  $q \leq p$ , cada  $Pr_i$  es un puntero de datos, y  $P_{sig}$  apunta al siguiente *nodo hoja* del árbol B<sup>+</sup>.
2. Dentro de cada nodo hoja,  $K_1 \leq K_2 \dots, K_{q-1}$ ,  $q \leq p$ .
3. Cada  $Pr_i$  es un **puntero a datos** que apunta al registro cuyo valor del campo de búsqueda es  $K_i$  o a un bloque del fichero que contiene el registro (o a un bloque de punteros de registro que apuntan a los registros cuyo valor del campo de búsqueda es  $K_i$  si el campo de búsqueda no es una clave).

<sup>9</sup> Nuestra definición obedece a Knuth (1973). Un árbol B<sup>+</sup> puede definirse de forma diferente intercambiando los símbolos  $< y \leq$  ( $K_{i-1} \leq X < K_i$ ;  $K_{q-1} \leq X$ ), pero los principios siguen siendo los mismos.

4. Cada nodo hoja tiene como mínimo  $\lceil (p/2) \rceil$  valores.

5. Todos los nodos hoja están en el mismo nivel.

Los punteros en los nodos internos son *punteros de árbol* a bloques que son nodos del árbol, mientras que los punteros de los nodos hoja son *punteros de datos* a registros o bloques del fichero (excepto para el puntero  $P_{sig}$ , que es un puntero de árbol al siguiente nodo hoja). Empezando por el nodo hoja más a la izquierda, es posible atravesar los nodos hoja como una lista enlazada, utilizando los punteros  $P_{sig}$ . Esto ofrece un acceso ordenado a los registros de datos del campo de indexación. También podemos incluir un puntero  $P_{anterior}$ . Para un árbol  $B^+$  en un campo no clave, se necesita un nivel de indirección adicional parecido al de la Figura 14.5, de tal manera que los punteros  $P_r$  son punteros de bloque a bloques que contienen un conjunto de punteros de registro a los registros actuales del fichero de datos, como explicamos en la Opción 3 de la Sección 14.1.3.

Como las entradas de los nodos internos de un árbol  $B^+$  incluyen valores de búsqueda y punteros de árbol sin punteros a los datos, es posible empaquetar más entradas en un nodo interno de un árbol  $B^+$  que en un árbol  $B$  similar. Por tanto, si el tamaño de bloque (nodo) es el mismo, el orden  $p$  será mayor para el árbol  $B^+$  que para el árbol  $B$ , como se ilustra en el Ejemplo 6. Esto puede reducir el número de niveles del árbol  $B^+$ , mejorándose así el tiempo de búsqueda. Como las estructuras de los nodos internos y de los nodos hoja de los árboles  $B^+$  son diferentes, su orden  $p$  puede ser diferente. Utilizaremos  $p$  para denotar el orden de los *nodos internos* y  $p_{hoja}$  para denotar el orden de los *nodos hoja*, que definimos como el número máximo de punteros a datos en un nodo hoja.

**Ejemplo 6.** Para calcular el orden  $p$  de un árbol  $B^+$ , vamos a suponer que el campo clave de búsqueda tiene una longitud de  $V = 9$  bytes, que el tamaño del bloque es de  $B = 512$  bytes, que un puntero de registro es de  $Pr = 7$  bytes, y que un puntero de bloque es de  $P = 6$  bytes, como en el Ejemplo 4. Un nodo interno del árbol  $B^+$  puede tener hasta  $p$  punteros de árbol y  $p - 1$  valores del campo de búsqueda; éstos deben encajar en un solo bloque. Por tanto, tenemos:

$$\begin{aligned}(p * P) + ((p - 1) * V) &\leq B \\(p * 6) + ((p - 1) * 9) &\leq 512 \\(15 * p) &\leq 521\end{aligned}$$

Podemos elegir  $p$  para que sea el valor más grande que satisfaga la desigualdad anterior, que da  $p = 34$ . Se trata de un valor mayor que el obtenido para un árbol  $B$  (23), lo que ofrece un *fan-out* más grande y más entradas en cada nodo interno de un árbol  $B^+$  que en el correspondiente árbol  $B$ . Los nodos hoja del árbol  $B^+$  tendrán el mismo número de valores y punteros, excepto que los punteros son punteros a datos y un puntero “siguiente”. Por tanto, el orden  $p_{hoja}$  para los nodos hoja puede calcularse de este modo:

$$\begin{aligned}(p_{hoja} * (Pr + V)) + P &\leq B \\(p_{hoja} * (7 + 9)) + 6 &\leq 512 \\(16 * p_{hoja}) &\leq 506\end{aligned}$$

Resulta que cada nodo hoja puede albergar hasta  $p_{hoja} = 31$  combinaciones de puntero valor/datos, asumiendo que los punteros de datos son punteros de registro.

Como con el árbol  $B$ , necesitamos información adicional (para implementar los algoritmos de inserción y eliminación) en cada nodo. Esta información puede incluir el tipo de nodo (interno u hoja), el número actual de entradas  $q$  en el nodo y punteros a los nodos padre y hermanos. Por tanto, antes de que hagamos los cálculos anteriores para  $p$  y  $p_{hoja}$ , debemos reducir el tamaño del bloque en la cantidad de espacio necesario para toda esta información. El siguiente ejemplo ilustra cómo podemos calcular el número de entradas en un árbol  $B^+$ .

**Ejemplo 7.** Supongamos que construimos un árbol  $B^+$  en el campo del Ejemplo 6. Para calcular el número aproximado de entradas del árbol  $B^+$ , asumimos que cada nodo está ocupado al 69%. Por término medio, cada nodo interno tendrá  $34 * 0,69$  o aproximadamente 23 punteros, y por tanto 22 valores. Cada nodo hoja, por

término medio, albergará  $0,69 * p_{\text{hoja}} = 0,69 * 31$  o aproximadamente 21 punteros a registros de datos. Un árbol B<sup>+</sup> tendrá por término medio la siguiente cantidad de entradas en cada nivel:

|             |              |                                      |                 |
|-------------|--------------|--------------------------------------|-----------------|
| Raíz:       | 1 nodo       | 22 entradas                          | 23 punteros     |
| Nivel 1:    | 23 nodos     | 506 entradas                         | 529 punteros    |
| Nivel 2:    | 529 nodos    | 11.638 entradas                      | 12.167 punteros |
| Nivel hoja: | 12.167 nodos | 255.507 punteros a registro de datos |                 |

Para un tamaño de bloque, tamaño de puntero y tamaño de campo de búsqueda dados, un árbol B<sup>+</sup> de tres niveles alberga hasta 255.507 punteros de registro, por término medio. Compare esto con las 65.535 entradas para el árbol B correspondiente del Ejemplo 5.

**Búsqueda, inserción y eliminación con árboles B<sup>+</sup>.** El Algoritmo 14.2 muestra el procedimiento utilizando el árbol B<sup>+</sup> como estructura de acceso para buscar un registro. El Algoritmo 14.3 ilustra el procedimiento para insertar un registro en un fichero con un árbol B<sup>+</sup> como estructura de acceso. Estos algoritmos asumen la existencia de un campo clave de búsqueda, y deben modificarse apropiadamente para el caso de un árbol B<sup>+</sup> en un campo no clave. Ilustramos la inserción y la eliminación con un ejemplo.

**Algoritmo 14.2. Búsqueda de un registro con un valor  $K$  del campo clave de búsqueda, utilizando un árbol B<sup>+</sup>.**

```

 $n \leftarrow$  bloque que contiene la raíz del árbol B+;
leer el bloque  $n$ ;
while ( $n$  no sea un nodo hoja del árbol B+) do
  begin
     $q \leftarrow$  número de punteros de árbol en el nodo  $n$ ;
    if  $K \leq n.K_1$  (* $n.K_i$  se refiere al valor del campo de búsqueda  $i$  en el nodo  $n$ *)
      then  $n \leftarrow n.P_1$  (* $n.P_i$  se refiere al puntero de árbol  $i$  en el nodo  $n$ *)
    else if  $K > n.K_{q-1}$ 
      then  $n \leftarrow n.P_q$ 
    else begin
      buscar el nodo  $n$  para una entrada  $i$  tal que  $n.K_{i-1} < K \leq n.K_i$ ;
       $n \leftarrow n.P_i$ 
    end;
  leer el bloque  $n$ 
end;
buscar en el bloque  $n$  la entrada  $(K_i, Pr_i)$  con  $K = K_i$ ; (* buscar nodo hoja *)
if encontrado
  then leer el bloque del fichero de datos con la dirección  $Pr_i$  y recuperar el registro
  else el registro con el valor de campo de búsqueda  $K$  no está en el fichero de datos;
```

**Algoritmo 14.3. Inserción de un registro con un valor de campo clave de búsqueda  $K$  en un árbol B<sup>+</sup> de orden  $p$ .**

```

 $n \leftarrow$  bloque que contiene la raíz del árbol B+;
leer el bloque  $n$ ; establecer la pila  $S$  a vacío;
while ( $n$  no sea un nodo hoja del árbol B+) do
  begin
    push la dirección de  $n$  a la pila  $S$ ;
    (*la pila  $S$  alberga los nodos padre necesarios en el caso de división*)
     $q \leftarrow$  número de punteros de árbol en el nodo  $n$ ;
```



```

if  $K \leq n.K_1$  (* $n.K_i$  se refiere al valor del campo de búsqueda  $i$  en el nodo  $n$ *)
then  $n \leftarrow n.P_1$  (* $n.P_i$  se refiere al puntero de árbol  $i$  en el nodo  $n$ *)
else if  $K > n.K_{q-1}$ 
then  $n \leftarrow n.P_q$ 
else begin
    buscar el nodo  $n$  para una entrada  $i$  tal que  $n.K_{i-1} < K \leq n.K_i$ ;
     $n \leftarrow n.P_i$ 
end;
leer el bloque  $n$ 
end;
buscar en el bloque  $n$  la entrada  $(K_i, Pr_i)$  con  $K = K_i$ ; (*buscar el nodo hoja  $n$ *)
if encontrado
then el registro ya existe en el fichero; no se puede insertar
else (*insertar la entrada en el árbol  $B^+$  para apuntar al registro*)
begin
    crear la entrada  $(K, Pr)$  donde  $Pr$  apunta al registro nuevo;
    if el nodo hoja  $n$  no está lleno
    then insertar la entrada  $(K, Pr)$  en la posición correcta en el nodo hoja  $n$ 
    else begin (*el nodo hoja  $n$  está lleno con  $p_{\text{hoja}}$  punteros de registro; se divide*)
        copiar  $n$  en  $temp$  (* $temp$  es un nodo hoja muy grande para almacenar la entrada extra*);
        insertar la entrada  $(K, Pr)$  en la posición correcta de  $temp$ ;
        (* $temp$  alberga ahora  $p_{\text{hoja}} + 1$  entradas en forma de  $(K_i, Pr_i)$ *)
         $nuevo \leftarrow$  un nuevo nodo hoja vacío para el árbol;  $nuevo.P_{\text{sig}} \leftarrow n.P_{\text{sig}}$ ;
         $j \leftarrow \lceil (p_{\text{hoja}} + 1)/2 \rceil$ ;
         $n \leftarrow$  primeras  $j$  entradas de  $temp$  (hasta la entrada  $(K_j, Pr_j)$ );  $n.P_{\text{sig}} \leftarrow nuevo$ ;
         $nuevo \leftarrow$  entradas restantes en  $temp$ ;  $K \leftarrow K_j$ ;
        (*ahora debemos mover  $(K, nuevo)$  e insertar en el nodo padre
        interno; sin embargo, si el padre está lleno, la división se puede propagar*)
        finalizado  $\leftarrow$  falso;
        repeat
        if la pila  $S$  está vacía
        then (*no hay nodo padre; se crea un nuevo nodo raíz para el árbol*)
            begin
                 $root \leftarrow$  un nuevo nodo interno vacío para el árbol;
                 $root \leftarrow \langle n, K, nuevo \rangle$ ; finalizado  $\leftarrow$  verdad;
            end
        else begin
             $n \leftarrow$  pop pila  $S$ ;
            if nodo interno  $n$  no está lleno
            then
                begin (*el nodo padre no está lleno; no hay división*)
                    insertar  $(K, nuevo)$  en la posición correcta en el nodo interno  $n$ ;
                    finalizado  $\leftarrow$  verdad
                end
            else begin (*el nodo interno  $n$  está lleno con  $p$  punteros de árbol; hay división*)
                copiar  $n$  en  $temp$  (* $temp$  es un nodo interno bastante grande*);
                insertar  $(K, nuevo)$  en  $temp$  en la posición correcta;
                (* $temp$  tiene ahora  $p + 1$  punteros de árbol*)

```

```

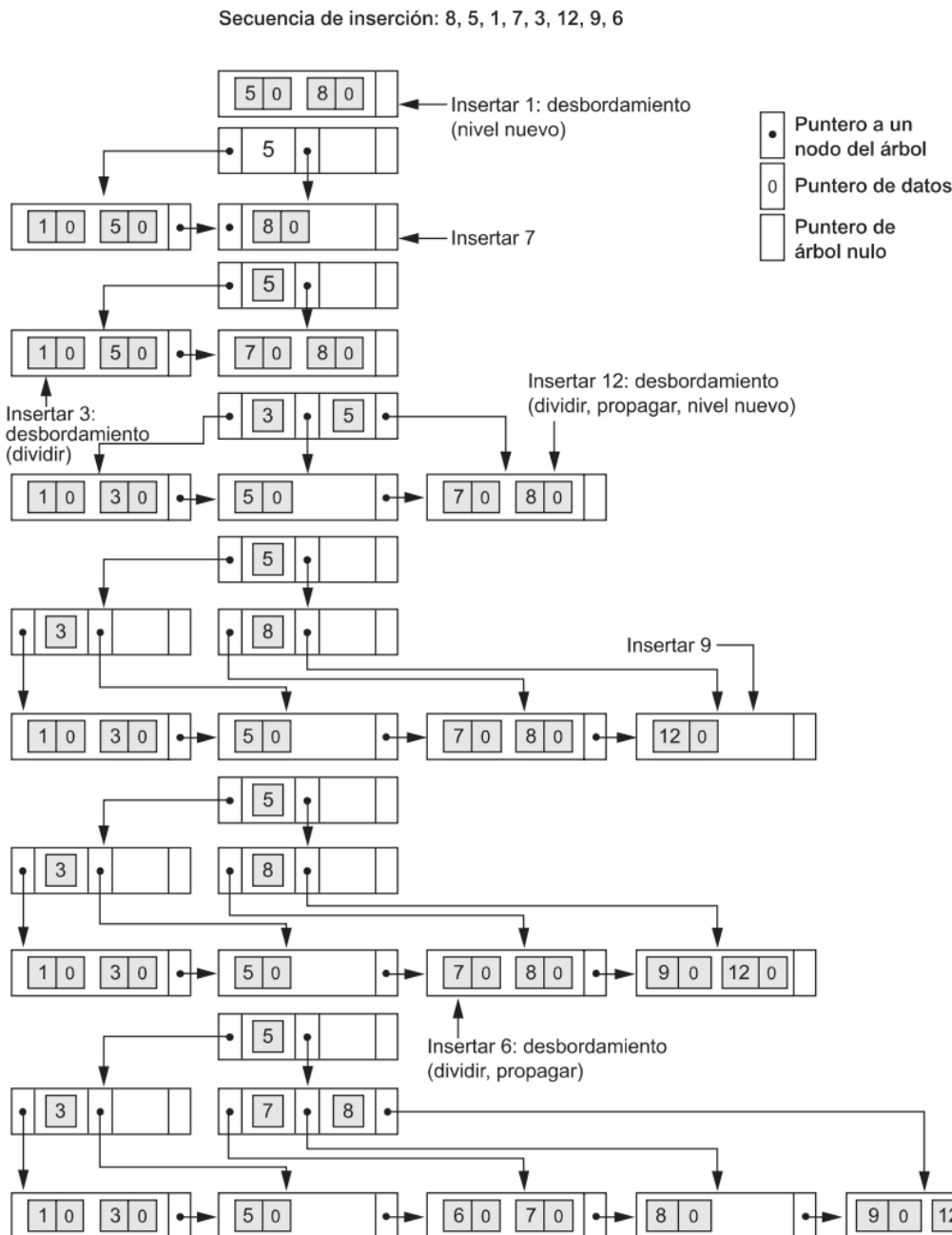
    nuevo ← un nuevo nodo interno vacío para el árbol;
     $j \leftarrow \lceil ((p + 1)/2) \rceil$ ;
     $n \leftarrow$  entradas hasta el puntero de árbol  $P_j$  en  $temp$ ;
    (* $n$  contiene  $\langle P_1, K_1, P_2, K_2, \dots, P_{j-1}, K_{j-1}, P_j \rangle$ *)
    nuevo ← entradas del puntero de árbol  $P_{j+1}$  en  $temp$ ;
    (* $nuevo$  contiene  $\langle P_{j+1}, K_{j+1}, \dots, K_{p-1}, P_p, K_p, P_{p+1} \rangle$ *)
     $K \leftarrow K_j$ 
    (*ahora debemos mover  $(K, nuevo)$  e insertar en el nodo interno
    padre*)
  end
end
until finalizado
end;
end;
```

La Figura 14.12 ilustra la **inserción** de registros en un árbol B<sup>+</sup> de orden  $p = 3$  y  $p_{\text{hoja}} = 2$ . En primer lugar, observamos que la raíz es el único nodo del árbol, por lo que también es un nodo hoja. En cuanto se crea más de un nivel, el árbol se divide en nodos internos y nodos hoja. Observe que **cada valor clave debe existir en el nivel hoja, porque todos los punteros de datos están en el nivel hoja**. Sin embargo, en los nodos internos sólo existen algunos valores para guiar la búsqueda. Además, cada valor que aparece en un nodo interno también aparece como **el valor más a la derecha** en el nivel hoja del subárbol apuntado por el puntero de árbol a la izquierda del valor.

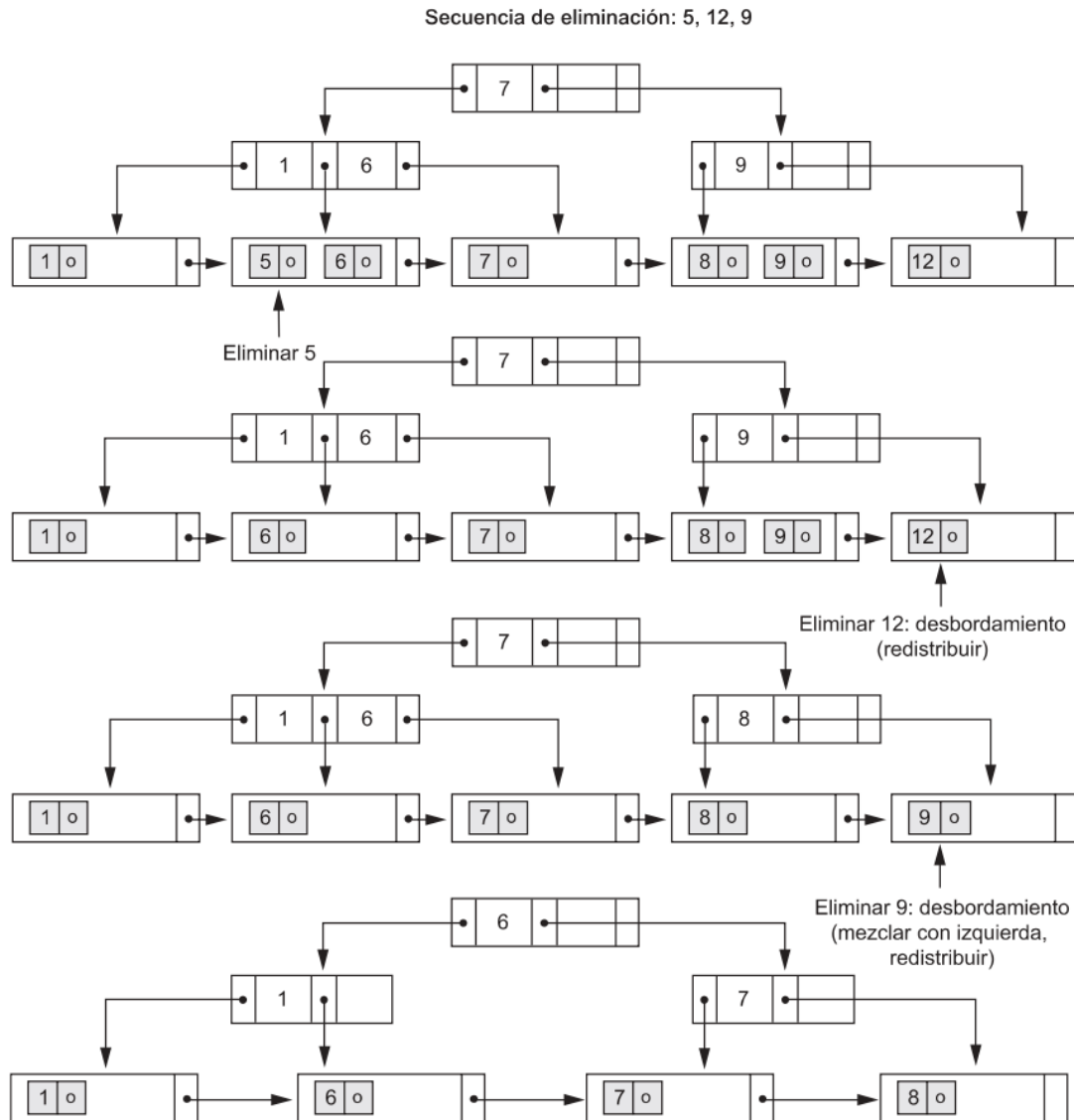
Cuando un **nodo hoja** está lleno y se inserta en él una entrada nueva, el nodo se desborda y debe dividirse. Las primeras  $j = \lceil ((p_{\text{hoja}} + 1)/2) \rceil$  entradas del nodo original se mantienen, mientras que el resto se mueve a un nuevo nodo hoja. El valor de búsqueda en la posición  $j$  se duplica en el nodo interno padre, y en éste se crea un puntero extra al nodo nuevo. Deben insertarse en el nodo padre en su secuencia correcta. Si el nodo interno padre está lleno, el valor nuevo también provocará su desbordamiento, por lo que deberá dividirse. Las entradas del nodo interno hasta  $P_j$  (el puntero de árbol número  $j$  después de insertar el valor y el puntero nuevos, donde  $j = \lceil ((p + 1)/2) \rceil$ ) se mantienen, mientras que el valor de búsqueda número  $j$  se mueve al padre, no se duplica. Un nuevo nodo interno albergará las entradas desde  $P_{j+1}$  hasta el final de las entradas del nodo (consulte el Algoritmo 14.3). Esta división puede propagarse hasta crear un nuevo nodo raíz y, por tanto, un nuevo nivel para el árbol B<sup>+</sup>.

La Figura 14.13 ilustra la **eliminación** en un árbol B<sup>+</sup>. Cuando se borra una entrada, **siempre se elimina del nivel hoja**. Si ocurre en un nodo interno, también debe eliminarse de allí. En este último caso, el valor a su izquierda en el nodo hoja debe sustituirse en el nodo interno porque este valor es ahora la entrada más a la derecha en el subárbol. La eliminación puede provocar un **desbordamiento por abajo**, al reducirse el número de entradas del nodo hoja por debajo del mínimo requerido. En este caso, intentamos encontrar un **nodo hoja hermano** (un nodo hoja situado inmediatamente a la izquierda o a la derecha del nodo desbordado por debajo) y redistribuir las entradas entre el nodo y su **hermano**, de modo que ambos estén rellenos, como mínimo, a la mitad; en caso contrario, el nodo se combina con sus hermanos y se reduce el número de nodos hoja. Un método común es intentar **redistribuir** las entradas con el hermano de la izquierda; si no es posible, se realiza un intento de redistribución con el hermano de la derecha. Si tampoco esto es posible, los tres nodos se combinan en dos nodos hoja. En tal caso, el desbordamiento por debajo se puede propagar a los nodos **internos** porque se necesitan un valor de búsqueda y un puntero de árbol menos. Esto puede propagarse y reducir los niveles del árbol.

La implementación de los algoritmos de inserción y eliminación puede requerir punteros padre y hermano para cada nodo, o el uso de una pila como la del Algoritmo 14.3. Cada nodo también debe incluir el número de entradas que tiene y su tipo (hoja o interno). Otra alternativa es implementar la inserción y la eliminación como procedimientos recursivos.

**Figura 14.12.** Ejemplo de inserción en un árbol B<sup>+</sup> con  $p = 3$  y  $p_{\text{hoja}} = 2$ .

**Variaciones de los árboles B y B<sup>+</sup>.** Para concluir esta sección, mencionaremos brevemente algunas variantes de los árboles B y B<sup>+</sup>. En algunos casos, la restricción 5 del árbol B (o del árbol B<sup>+</sup>), que requiere que cada nodo esté lleno como mínimo hasta la mitad, puede modificarse para que **cada nodo esté lleno a sus dos tercios**. En este caso, el árbol B se denomina **árbol B\***. En general, algunos sistemas permiten al usuario elegir un **factor de relleno** entre 0,5 y 1,0, donde esto último significa que los nodos del árbol B (índice) están completamente llenos. También es posible especificar dos factores de relleno para un árbol B<sup>+</sup>: uno para el

**Figura 14.13.** Ejemplo de eliminación en un árbol B<sup>+</sup>.

nivel hoja y otro para los nodos internos del árbol. Cuando se construye el índice al principio, los nodos se rellenan hasta aproximadamente los factores de relleno especificados. Recientemente, los investigadores han sugerido relajar el requisito de que un nodo esté lleno hasta la mitad, y en su lugar permiten que un nodo se quede completamente vacío antes de combinarse, simplificándose así el algoritmo de eliminación. Los estudios de simulación muestran que esto no desperdicia demasiado espacio adicional bajo inserciones y eliminaciones aleatoriamente distribuidas.

## 14.4 Índices en claves múltiples

Hasta ahora, en nuestra explicación hemos asumido que las claves principal y secundaria con las que se accede a los ficheros son atributos sencillos (campos). En muchas solicitudes de recuperación y actualización, son